```
import sqlite3 import numpy as np import pandas as pd from datetime import datetime, timedelta import random import os np.random.seed(42) random.seed(42) DB_PATH = "swiftride.db" if os.path.exists(DB_PATH): os.remove(DB_PATH) conn = sqlite3.connect(DB_PATH) cursor = conn.cursor() cursor.executescript(""" PRAGMA foreign_keys = ON; CREATE TABLE cities ( city_id INTEGER PRIMARY KEY, city_name TEXT NOT NULL, province TEXT NOT NULL, population INTEGER NOT NULL, is_active INTEGER NOT NULL DEFAULT 1 ); CREATE TABLE drivers ( driver_id INTEGER PRIMARY KEY, name TEXT NOT NULL, phone TEXT NOT NULL, city_id INTEGER NOT NULL, vehicle_type TEXT NOT NULL, vehicle_model TEXT NOT NULL, rating REAL NOT NULL, total_trips INTEGER NOT NULL DEFAULT 0, is_active INTEGER NOT NULL DEFAULT 1, joined_date TEXT NOT NULL, FOREIGN KEY (city_id) REFERENCES cities(city_id) ); CREATE TABLE riders ( rider_id INTEGER PRIMARY KEY, name TEXT NOT NULL, phone TEXT NOT NULL, email TEXT NOT NULL, city_id INTEGER NOT NULL, signup_date TEXT NOT NULL, total_trips INTEGER NOT NULL DEFAULT 0, rating REAL NOT NULL, FOREIGN KEY (city_id) REFERENCES cities(city_id) ); CREATE TABLE trips ( trip_id INTEGER PRIMARY KEY, rider_id INTEGER NOT NULL, driver_id INTEGER NOT NULL, city_id INTEGER NOT NULL, pickup_area TEXT NOT NULL, dropoff_area TEXT NOT NULL, vehicle_type TEXT NOT NULL, distance_km REAL NOT NULL, duration_mins REAL NOT NULL, fare_pkr REAL NOT NULL, trip_date TEXT NOT NULL, trip_hour INTEGER NOT NULL, day_of_week INTEGER NOT NULL, status TEXT NOT NULL, is_raining INTEGER NOT NULL DEFAULT 0, is_peak_hour INTEGER NOT NULL DEFAULT 0, surge_multiplier REAL NOT NULL DEFAULT 1.0, FOREIGN KEY (rider_id) REFERENCES riders(rider_id), FOREIGN KEY (driver_id) REFERENCES drivers(driver_id), FOREIGN KEY (city_id) REFERENCES cities(city_id) ); CREATE TABLE payments ( payment_id INTEGER PRIMARY KEY, trip_id INTEGER NOT NULL, amount_pkr REAL NOT NULL, payment_method TEXT NOT NULL, payment_status TEXT NOT NULL, paid_at TEXT NOT NULL, FOREIGN KEY (trip_id) REFERENCES trips(trip_id) ); CREATE TABLE reviews ( review_id INTEGER PRIMARY KEY, trip_id INTEGER NOT NULL, rider_id INTEGER NOT NULL, driver_id INTEGER NOT NULL, rider_rating_given REAL NOT NULL, driver_rating_given REAL NOT NULL, comment_sentiment TEXT NOT NULL, review_date TEXT NOT NULL, FOREIGN KEY (trip_id) REFERENCES trips(trip_id), FOREIGN KEY (rider_id) REFERENCES riders(rider_id), FOREIGN KEY (driver_id) REFERENCES drivers(driver_id) ); """) conn.commit() # ——————————————————————————————————————— # 1. CITIES # ——————————————————————————————————————— print("Creating cities...") cities_data = [ (1, "Karachi", "Sindh", 16_000_000, 1), (2, "Lahore", "Punjab", 13_000_000, 1), (3, "Islamabad", "Islamabad Capital Territory", 1_100_000, 1), (4, "Rawalpindi", "Punjab", 2_300_000, 1), (5, "Peshawar", "Khyber Pakhtunkhwa", 2_100_000, 1), (6, "Quetta", "Balochistan", 1_200_000, 1), (7, "Multan", "Punjab", 1_900_000, 1), (8, "Faisalabad", "Punjab", 3_600_000, 1), ] cursor.executemany( "INSERT INTO cities VALUES (?,?,?,?,?)", cities_data ) conn.commit() print(f" → {len(cities_data)} cities inserted.") # ——————————————————————————————————————— # 2. HELPERS — names, areas, vehicles # ——————————————————————————————————————— MALE_FIRST = [ "Ahmed", "Ali", "Muhammad", "Hassan", "Usman", "Bilal", "Faisal", "Tariq", "Imran", "Zubair", "Adnan", "Kashif", "Shahid", "Waqas", "Naveed", "Kamran", "Salman", "Omer", "Rizwan", "Asif", "Fahad", "Zeeshan", "Tahir", "Jawad", "Saad", "Hamza", "Shoaib", "Danish", "Babar", "Irfan", "Wasim", "Khalid", "Nasir", "Sohail", "Amir", "Rehan", "Furqan", "Arslan", "Raza", "Ijaz", "Qasim", "Anas", "Farhan", "Taha", "Noman", "Yasir", "Shehzad", "Atif", "Ehsan", "Waleed", ] FEMALE_FIRST = [ "Ayesha", "Fatima", "Zainab", "Maryam", "Sana", "Hira", "Nadia", "Amna", "Sara", "Rabia", "Bushra", "Samina", "Nabeela", "Iqra", "Sidra", "Maham", "Noor", "Rida", "Asma", "Saima", "Uzma", "Farah", "Mehwish", "Shazia", "Rukhsana", "Khadija", "Fiza", "Anum", "Aroha", "Laiba", ] LAST_NAMES = [ "Khan", "Ahmed", "Ali", "Sheikh", "Malik", "Chaudhry", "Butt", "Siddiqui", "Akhtar", "Mirza", "Qureshi", "Hussain", "Baig", "Rana", "Raza", "Nawaz", "Abbasi", "Ansari", "Javed", "Iqbal", "Rashid", "Hashmi", "Niazi", "Baloch", "Afridi", "Yousaf", "Bhatti", "Gondal", "Asghar", "Rizvi", ] CITY_AREAS = { 1: ["DHA", "Clifton", "Gulshan-e-Iqbal", "Saddar", "Korangi", "Nazimabad", "Malir", "Surjani", "North Nazimabad", "Liaquatabad", "Federal B Area", "Scheme 33", "Bahria Town", "Gulistan-e-Johar", "Landhi"], 2: ["DHA Lahore",
```

```
"Gulberg", "Model Town", "Johar Town", "Bahria Town", "Cantt", "Iqbal Town", "Wapda Town", "Garden Town", "Faisal Town", "Valencia", "Raiwind Road", "Shadman", "Township", "Samanabad"], 3: ["F-7", "F-8", "F-10", "F-11", "G-9", "G-10", "G-11", "I-8", "I-10", "Blue Area", "E-7", "Bahria Town Islamabad", "DHA Islamabad", "Bani Gala", "Margalla Hills"], 4: ["Saddar Rawalpindi", "Bahria Town Rawalpindi", "Chaklala", "Satellite Town", "Gulraiz", "Dhoke Hassu", "Westridge", "Chandni Chowk", "Lalazar", "Raja Bazaar"], 5: ["Hayatabad", "University Town", "Saddar Peshawar", "Cantt Peshawar", "Dalazak Road", "Kohat Road", "Regi", "Gulbahar", "Tehkal", "Pakha Ghulam"], 6: ["Satellite Town Quetta", "Jinnah Town", "Sariab Road", "Brewery Road", "Shalkot", "Airport Road Quetta", "Samungli Road", "Zarghoon Road", "Mezan Chowk", "Spini Road"], 7: ["Cantt Multan", "Shah Rukn-e-Alam", "Gulgasht Colony", "New Multan", "Bosan Road", "Qasim Bela", "Chungi No 9", "Hussain Agahi", "Vehari Road", "Abdali Road"], 8: ["Peoples Colony", "Millat Road", "Ghulam Muhammad Abad", "Madina Town", "Jinnah Colony", "Canal Road Faisalabad", "Dijkot Road", "Satiana Road", "D Ground", "Kohinoor City"], } VEHICLE_MODELS = { "Bike": ["Honda CD 70", "Yamaha YBR 125", "Honda CG 125", "Suzuki GS 150", "Honda CB 150F", "Road Prince 70"], "Rickshaw": ["Qingqi 100cc", "Ravi Piaggio", "Loader Rickshaw", "CNG Rickshaw", "Electric Rickshaw"], "Car": ["Suzuki Alto", "Toyota Corolla", "Honda City", "Suzuki Cultus", "Hyundai Tucson", "KIA Picanto", "Toyota Yaris", "Suzuki Wagon R", "Honda Civic"], "SUV": ["Toyota Fortuner", "Toyota Prado", "Honda BR-V", "Hyundai Tucson AWD", "KIA Sportage", "Toyota Hilux", "Isuzu D-Max"], } PEAK_HOURS = set(range(7, 10)) | set(range(17, 21)) # 7-9, 17-20 def random_pk_phone(): return "03" + str(np.random.randint(0, 5)) + \ "".join([str(np.random.randint(0, 10)) for _ in range(8)]) def random_name(female_prob=0.30): if np.random.random() < female_prob: first = np.random.choice(FEMALE_FIRST) else: first = np.random.choice(MALE_FIRST) last = np.random.choice(LAST_NAMES) return f"{first} {last}" def random_date_str(start: str, end: str) -> str: s = datetime.strptime(start, "%Y-%m-%d") e = datetime.strptime(end, "%Y-%m-%d") delta = (e - s).days return (s + timedelta(days=int(np.random.randint(0, delta + 1)))).strftime("%Y-%m-%d") # ──────────────────────────────────────────────────────────── # 3. DRIVERS (150 total) # ────────────────────────────────────────────────────────────
print("Creating drivers...") # City weights proportional to population / ride-share penetration CITY_DRIVER_WEIGHTS = np.array([30, 28, 15, 12, 18, 8, 17, 22], dtype=float) CITY_DRIVER_WEIGHTS /= CITY_DRIVER_WEIGHTS.sum() city_ids = [c[0] for c in cities_data] # Vehicle type distribution: Bike 35%, Rickshaw 20%, Car 35%, SUV 10% VEHICLE_TYPES = ["Bike", "Rickshaw", "Car", "SUV"] VEHICLE_PROBS = [0.35, 0.20, 0.35, 0.10] drivers_rows = [] for driver_id in range(1, 151): city_id = int(np.random.choice(city_ids, p=CITY_DRIVER_WEIGHTS)) vtype = str(np.random.choice(VEHICLE_TYPES, p=VEHICLE_PROBS)) vmodel = str(np.random.choice(VEHICLE_MODELS[vtype])) rating = round(float(np.random.beta(8, 2) * 1.5 + 3.5), 2) # skewed high, 3.5-5.0 rating = min(5.0, max(3.5, rating)) total_trips = int(np.random.randint(10, 800)) is_active = 1 if np.random.random() < 0.85 else 0 joined_date = random_date_str("2022-01-01", "2024-06-01") name = random_name(female_prob=0.08) # mostly male drivers phone = random_pk_phone() drivers_rows.append((driver_id, name, phone, city_id, vtype, vmodel, rating, total_trips, is_active, joined_date)) cursor.executemany( "INSERT INTO drivers VALUES (?,?,?,?,?,?,?,?,?,?)", drivers_rows ) conn.commit() print(f" → {len(drivers_rows)} drivers inserted.") # ──────────────────────────────────────────────────────────── # 4. RIDERS (800 total) # ────────────────────────────────────────────────────────────
print("Creating riders...") CITY_RIDER_WEIGHTS = np.array([200, 180, 80, 70, 80, 30, 70, 90], dtype=float) CITY_RIDER_WEIGHTS /= CITY_RIDER_WEIGHTS.sum() riders_rows = [] for rider_id in range(1, 801): city_id = int(np.random.choice(city_ids, p=CITY_RIDER_WEIGHTS)) name = random_name(female_prob=0.40) phone = random_pk_phone() first_part = name.lower().replace(" ", ".") + str(np.random.randint(10, 999)) domains = ["gmail.com", "yahoo.com", "hotmail.com", "outlook.com"] email = first_part + "@" + np.random.choice(domains) signup_date = random_date_str("2022-06-01", "2024-12-01") total_trips = int(np.random.randint(1, 300)) rating = round(float(np.random.beta(6, 2) * 2.0 + 3.0), 2) rating = min(5.0, max(3.0, rating)) riders_rows.append((rider_id, name, phone, email, city_id, signup_date, total_trips, rating)) cursor.executemany( "INSERT INTO riders VALUES (?,?,?,?,?,?,?,?)", riders_rows ) conn.commit() print(f" → {len(riders_rows)} riders inserted.") # ──────────────────────────────────────────────────────────── # 5. TRIPS (7000 total)
```

```
# ──────────────────────────────────────────────────────────────────────── print("Creating trips... (this may take a moment)") # Build lookup: city_id -> list of active driver_ids and their vehicle type from collections import defaultdict city_active_drivers = defaultdict(list) # city_id -> [(driver_id, vtype)] for row in drivers_rows: did, _, _, cid, vtype, _, _, _, is_active, _ = row if is_active: city_active_drivers[cid].append((did, vtype)) city_active_riders = defaultdict(list) # city_id -> [rider_id] for row in riders_rows: rid, _, _, _, cid, *_ = row city_active_riders[cid].append(rid) # City trip distribution: Karachi(1) + Lahore(2) = 35%, rest proportional CITY_TRIP_WEIGHTS = { 1: 0.20, # Karachi 2: 0.15, # Lahore 3: 0.12, # Islamabad 4: 0.10, # Rawalpindi 5: 0.11, # Peshawar 6: 0.07, # Quetta 7: 0.10, # Multan 8: 0.15, # Faisalabad } city_trip_ids = list(CITY_TRIP_WEIGHTS.keys()) city_trip_prob = np.array(list(CITY_TRIP_WEIGHTS.values())) city_trip_prob /= city_trip_prob.sum() STATUS_CHOICES = ["completed", "cancelled", "no_show"] STATUS_PROBS = [0.92, 0.05, 0.03] DISTANCE_RANGES = { "Bike": (1.0, 8.0), "Rickshaw": (1.0, 6.0), "Car": (2.0, 20.0), "SUV": (3.0, 25.0), } BASE_FARE = { "Bike": (50, 25), "Rickshaw": (60, 30), "Car": (100, 45), "SUV": (150, 60), } # Hour distribution: weighted toward commute peaks and evening HOUR_WEIGHTS = np.array([ 0.5, 0.3, 0.2, 0.2, 0.3, 0.5, # 0-5 late night / early morning 1.0, 2.5, 3.0, 2.0, 1.5, 1.2, # 6-11 morning 1.0, 0.9, 0.8, 1.0, 1.5, 2.8, # 12-17 3.2, 3.0, 2.5, 2.0, 1.5, 1.0, # 18-23 ]) HOUR_WEIGHTS /= HOUR_WEIGHTS.sum() # Date range: 2023-01-01 to 2024-12-31 (730 days) TRIP_START = datetime(2023, 1, 1) TRIP_END = datetime(2024, 12, 31) TOTAL_DAYS = (TRIP_END - TRIP_START).days + 1 # Seasonal weight per month (index 1=Jan … 12=Dec) # More trips in winter (Nov-Feb) and less in hot summer (May-Jul) MONTHLY_WEIGHT = { 1: 1.15, 2: 1.10, 3: 1.00, 4: 0.92, 5: 0.85, 6: 0.80, 7: 0.82, 8: 0.88, 9: 0.95, 10: 1.00, 11: 1.12, 12: 1.18, } # Pre-generate all trip dates with seasonal weighting all_dates = [TRIP_START + timedelta(days=i) for i in range(TOTAL_DAYS)] date_weights = np.array([MONTHLY_WEIGHT[d.month] for d in all_dates], dtype=float) date_weights /= date_weights.sum() chosen_date_indices = np.random.choice(len(all_dates), size=7000, p=date_weights) trips_rows = [] skipped = 0 for trip_id in range(1, 7001): city_id = int(np.random.choice(city_trip_ids, p=city_trip_prob)) # Ensure the city has active drivers and riders if not city_active_drivers[city_id] or not city_active_riders[city_id]: skipped += 1 # fallback to Karachi which always has coverage city_id = 1 driver_entry = random.choice(city_active_drivers[city_id]) driver_id, vtype = driver_entry rider_id = int(np.random.choice(city_active_riders[city_id])) # Date & time trip_date_obj = all_dates[chosen_date_indices[trip_id - 1]] trip_date = trip_date_obj.strftime("%Y-%m-%d") day_of_week = trip_date_obj.weekday() # 0=Mon, 6=Sun trip_hour = int(np.random.choice(range(24), p=HOUR_WEIGHTS)) is_peak_hour = 1 if trip_hour in PEAK_HOURS else 0 is_raining = 1 if np.random.random() < 0.15 else 0 # Areas areas = CITY_AREAS[city_id] pickup_area = np.random.choice(areas) dropoff_area = np.random.choice(areas) while dropoff_area == pickup_area and len(areas) > 1: dropoff_area = np.random.choice(areas) # Distance & duration dmin, dmax = DISTANCE_RANGES[vtype] distance_km = round(float(np.random.uniform(dmin, dmax)), 2) duration_mins = round(float(distance_km * 4 + np.random.uniform(-2, 6)), 1) duration_mins = max(3.0, duration_mins) # Fare base_fixed, per_km = BASE_FARE[vtype] base_fare = base_fixed + per_km * distance_km if is_raining and is_peak_hour: surge = 1.5 elif is_peak_hour: surge = 1.3 else: surge = 1.0 noise = np.random.uniform(-20, 20) fare_raw = base_fare * surge + noise fare_pkr = round(fare_raw / 10) * 10 # round to nearest 10 fare_pkr = max(30, fare_pkr) # Status status = str(np.random.choice(STATUS_CHOICES, p=STATUS_PROBS)) trips_rows.append(( trip_id, rider_id, driver_id, city_id, pickup_area, dropoff_area, vtype, distance_km, duration_mins, fare_pkr, trip_date, trip_hour, day_of_week, status, is_raining, is_peak_hour, round(surge, 2) )) cursor.executemany( "INSERT INTO trips VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)", trips_rows ) conn.commit() print(f" → {len(trips_rows)} trips inserted. ({skipped} city fallbacks used)") # ──────────────────────────────────────────────────────────────────────── # 6. PAYMENTS (one per completed trip) # ──────────────────────────────────────────────────────────────────────── print("Creating payments...") PAYMENT_METHODS = ["cash", "card", "wallet"] PAYMENT_PROBS = [0.60, 0.25, 0.15] PAY_STATUS = ["completed", "failed"] PAY_STATUS_P = [0.98, 0.02] completed_trips = [t for t in trips_rows if t[13] == "completed"] payments_rows = [] for pay_id, trip in enumerate(completed_trips, start=1): trip_id = trip[0] fare_pkr = trip[9] trip_date
```

```
= trip[10] method = str(np.random.choice(PAYMENT_METHODS, p=PAYMENT_PROBS)) pstatus = str(np.random.choice(PAY_STATUS, p=PAY_STATUS_P)) # paid_at = trip_date + random hour paid_hour = np.random.randint(0, 24) paid_min = np.random.randint(0, 60) paid_at = f"{trip_date} {paid_hour:02d}:{paid_min:02d}:00" payments_rows.append((pay_id, trip_id, fare_pkr, method, pstatus, paid_at)) cursor.executemany( "INSERT INTO payments VALUES (?,?,?,?,?,?)", payments_rows ) conn.commit() print(f" → {len(payments_rows)} payments inserted.") # ──────────────────────────────────────────────────────────── # 7. REVIEWS (65% of completed trips) # ──────────────────────────────────────────────────────────── print("Creating reviews...") SENTIMENT_CHOICES = ["positive", "neutral", "negative"] SENTIMENT_PROBS = [0.70, 0.20, 0.10] review_id = 1 reviews_rows = [] for trip in completed_trips: if np.random.random() > 0.65: continue trip_id = trip[0] rider_id = trip[1] driver_id = trip[2] trip_date = trip[10] # Rider rates the driver — skewed positive (3-5) rider_rating = float(np.random.choice( [1, 2, 3, 4, 5], p=[0.02, 0.04, 0.10, 0.28, 0.56] )) # Driver rates the rider — slightly more uniform driver_rating = float(np.random.choice( [1, 2, 3, 4, 5], p=[0.02, 0.03, 0.12, 0.33, 0.50] )) sentiment = str(np.random.choice(SENTIMENT_CHOICES, p=SENTIMENT_PROBS)) review_date = trip_date reviews_rows.append(( review_id, trip_id, rider_id, driver_id, rider_rating, driver_rating, sentiment, review_date )) review_id += 1 cursor.executemany( "INSERT INTO reviews VALUES (?,?,?,?,?,?,?,?)", reviews_rows ) conn.commit() print(f" → {len(reviews_rows)} reviews inserted.") # ──────────────────────────────────────────────────────────── # 8. SUMMARY # ──────────────────────────────────────────────────────────── print("\n" + "=" * 50) print(" SwiftRide DB — Generation Complete") print("=" * 50) tables = ["cities", "drivers", "riders", "trips", "payments", "reviews"] for table in tables: cursor.execute(f"SELECT COUNT(*) FROM {table}") count = cursor.fetchone()[0] print(f" {table:<12} : {count:>7,} rows") print("=" * 50) print(f" Database saved to: {os.path.abspath(DB_PATH)}") print("=" * 50) conn.close()
```